Computer Architecture
Instructions: Language of the Computer (I)

Tatpong Katanyukul

CoE, KKU

1 พฤษภาคม พ.ศ. 2560

Instruction Set

- Instruction set:
  The vocabulary of commands understood by a given architecture
  E.g., x86, ARMv7, ARMv8, MIPS 32
- Stored-program concept:
  The idea that instructions and data of many types can be stored in memory as numbers, leading to the stored-program computer.
- Computer designers have a common goal:
  to find a language that makes it easy to build the hardware and the compiler while maximizing performance and minimizing cost and energy.

The MIPS Instruction Set

- www.mips.com
- typical of many moderm ISAs

MIPS operands

Name	Example	Comments
32 registers	`$s0-$s7, $t0-$t9, $zero, $a0-$a3, $v0-$v1, $gp, $fp, $sp, $ra, $at`	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register `$zero` always equals 0, and register `$at` is reserved by the assembler to handle large constants.
2^{30} memory words	Memory[0], Memory[4], . . . , Memory[4294967292]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

The MIPS Instruction Set

Category	Instruction	Example
Arithmetic	add	`add $s1,$s2,$s3`
	subtract	`sub $s1,$s2,$s3`
	add immediate	`addi $s1,$s2,20`
Data transfer	load word	`lw $s1,20($s2)`
	store word	`sw $s1,20($s2)`
	load half	`lh $s1,20($s2)`
	load half unsigned	`lhu $s1,20($s2)`
	store half	`sh $s1,20($s2)`
	load byte	`lb $s1,20($s2)`
	load byte unsigned	`lbu $s1,20($s2)`
	store byte	`sb $s1,20($s2)`
	load linked word	`ll $s1,20($s2)`
	store condition. word	`sc $s1,20($s2)`
	load upper immed.	`lui $s1,20`
Logical	and	`and $s1,$s2,$s3`
	or	`or $s1,$s2,$s3`
	nor	`nor $s1,$s2,$s3`
	and immediate	`andi $s1,$s2,20`
	or immediate	`ori $s1,$s2,20`
	shift left logical	`sll $s1,$s2,10`
	shift right logical	`srl $s1,$s2,10`

Category	Instruction	Example
Conditional branch	branch on equal	`beq $s1,$s2,25`
	branch on not equal	`bne $s1,$s2,25`
	set on less than	`slt $s1,$s2,$s3`
	set on less than unsigned	`sltu $s1,$s2,$s3`
	set less than immediate	`slti $s1,$s2,20`
	set less than immediate unsigned	`sltiu $s1,$s2,20`
Unconditional jump	jump	`j 2500`
	jump register	`jr $ra`
	jump and link	`jal 2500`

Arithmetic Instructions

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three register operands
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three register operands
	add immediate	`addi $s1,$s2,20`	$s1 = $s2 + 20	Used to add constants

- Design Principle 1: Simplicity favors regularity.
- คำสั่งของ MIPS ต้องการ 3 operands ทุกคำสั่ง

Arithmetic Instructions

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three register operands
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three register operands
	add immediate	`addi $s1,$s2,20`	$s1 = $s2 + 20	Used to add constants

ภาษา C:

```
a = b + c;
d = a - e;
```

MIPS instructions:

```
add a, b, c
sub d, a, e
```

Compiling a Complex C Assignment into MIPS

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three register operands
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three register operands
	add immediate	`addi $s1,$s2,20`	$s1 = $s2 + 20	Used to add constants

ภาษา C:

```
f = (g + h) - (i + j);
```

MIPS instructions:

```
# คอมเมนต์
add t0, g, h   # t0 เป็น temporary variable
add t1, i, j
sub f, t0, t1
```

Operands of the Computer Hardware

- Arithmetic instructions use register operands.
- Register: hardware storing a word of data.
  เช่น ใช้ 32 D-flip-flops สร้างเป็น register ขนาด 32 บิต
- word (คอมพิวเตอร์): ขนาดของข้อมูลที่ทำการคำนวณในคอมพิวเตอร์
  เช่น MIPS ใช้ register เก็บข้อมูลสำหรับการคำนวณ และ register มีขนาด 32 บิต $\Rightarrow$ word = 32.

Registers

- MIPS มี ISA MIPS 32 และ MIPS 64
  เราเรียน MIPS 32 (ขนาดของ register คือ 32 บิต)
- MIPS มี 32-bit registers ให้ใช้ได้ 32 ตัว

Name	Register number	Usage
$zero	0	The constant value 0
$v0–$v1	2–3	Values for results and expression evaluation
$a0–$a3	4–7	Arguments
$t0–$t7	8–15	Temporaries
$s0–$s7	16–23	Saved
$t8–$t9	24–25	More temporaries
$gp	28	Global pointer
$sp	29	Stack pointer
$fp	30	Frame pointer
$ra	31	Return address

Registers

- programming variables มีจำนวนไม่จำกัด
  แต่จำนวน registers ใน hardware จำกัด
- MIPS จำกัดจำนวน registers ไว้แค่ 32 ตัว
- Design Principle 2: Smaller is faster.
  More registers $\Rightarrow$ larger circuit $\Rightarrow$ signals travel longer.
- 32 registers $\Rightarrow$ ต้องการ 5 บิตในการอ้างถึง (มีผลกับความยาวของ instruction)
- คำสั่งทุกคำสั่งใน MIPS ยาว 32 บิตเท่ากันหมด
  Simplicity favors regularity.
  $\Rightarrow$ ฮาร์ดแวร์ทำได้ง่ายกว่า

Registers

Name	Register number	Usage
$zero	0	The constant value 0
$v0–$v1	2–3	Values for results and expression evaluation
$a0–$a3	4–7	Arguments
$t0–$t7	8–15	Temporaries
$s0–$s7	16–23	Saved
$t8–$t9	24–25	More temporaries
$gp	28	Global pointer
$sp	29	Stack pointer
$fp	30	Frame pointer
$ra	31	Return address

- อ้างรีจิสเตอร์ อ้างเป็นเลขก็ได้

  add 17, 0, 0
- นิยมอ้างเป็นชื่อ

  add $s1, $zero, $zero

Register Operand Example

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three register operands
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three register operands
	add immediate	`addi $s1,$s2,20`	$s1 = $s2 + 20	Used to add constants

ภาษา C:

```
f = (g + h) - (i + j);
```

กำหนดให้ registes $s0, $s1, $s2, $s3, $s4 แทน variables f, g, h, i, j ตามลำดับ.

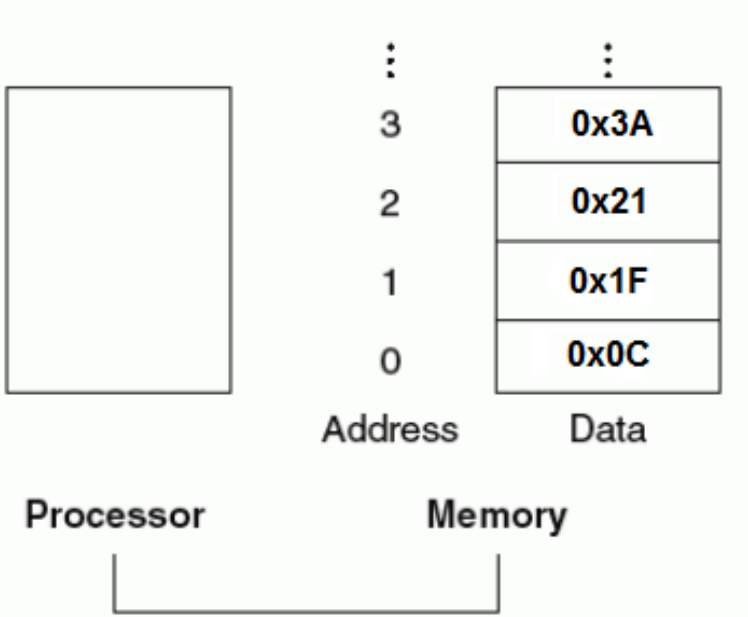
MIPS instructions:

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```

Memory Operands

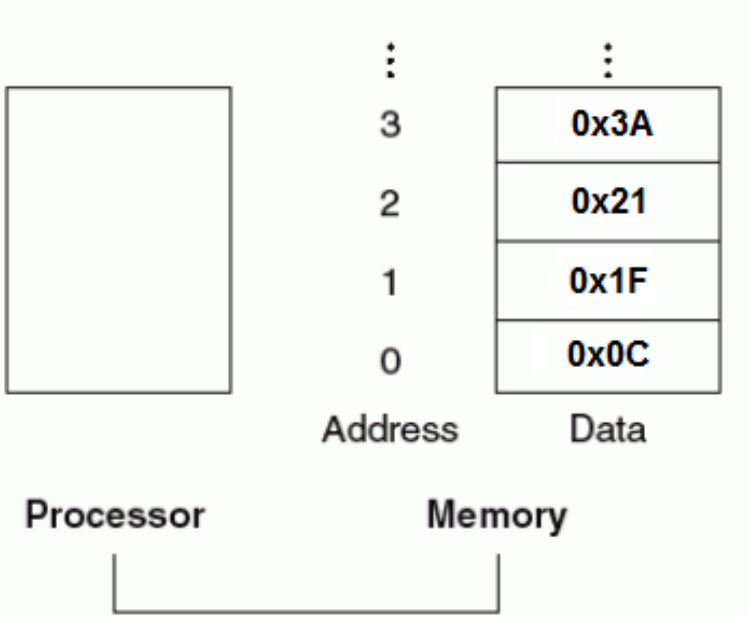
- Main memory ใช้กับ
  E.g., arrays, structures, dynamic data
- บางโปรแกรม ใช้ # variables $>$ # registers
  $\Rightarrow$ เก็บ variables ที่เกินใน memory
  "spilling registers"
- Memory เป็น byte addressed
  แต่ละ address เป็น ดัชนีอ้างถึง ข้อมูล 8 บิต ($\equiv$ 1 byte)

Big Endian v.s. Little Endian



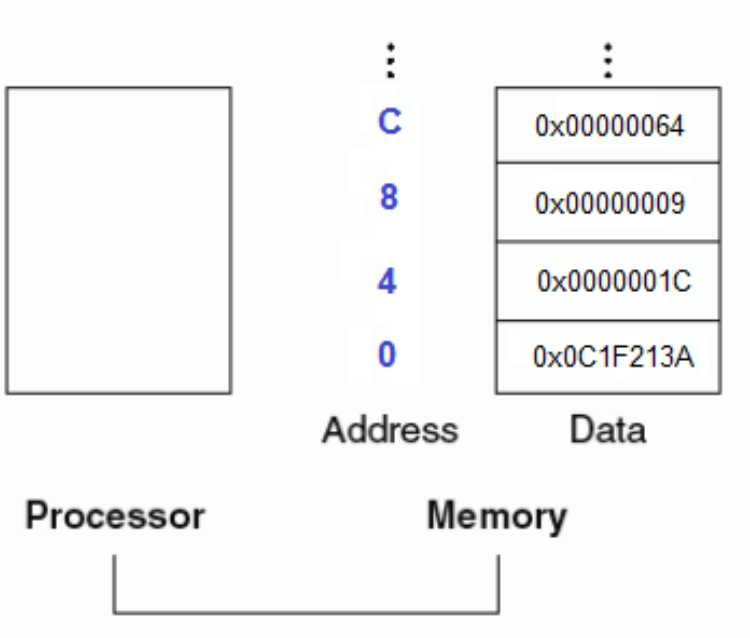
- Memory เป็น byte addressed: 8 bits
- MIPS 32 ใช้ ขนาด word เป็น 32 bits
  32 bits = 4 bytes
- ถ้าอ้าง memory ที่ address 0 จะอ่าน
  - 0x0C1F213A หรือ 0x3A211F0C ?

Big Endian v.s. Little Endian



- Memory เป็น byte addressed: 8 bits
- MIPS 32 ใช้ ขนาด word เป็น 32 bits
  32 bits = 4 bytes
- ถ้าอ้าง memory ที่ address 0 จะอ่าน
  - 0x0C1F213A หรือ 0x3A211F0C ?
  - Big Endian: 0x0C1F213A
  - Little Endian: 0x3A211F0C
  - MIPS เป็น Big Endian: 0x0C1F213A

Alignment Restriction



- word ของ MIPS จะขยับ address ทีละ 4
- alignment restriction: a requirement that data be aligned in memory on natural boundaries.

  E.g., อ้าง memory ที่ address

  0x 2000 0000 ได้

  0x 2000 0004 ได้

  0x 2000 0008 ได้

  0x 2000 000C ได้

  0x 2000 0010 ได้

  0x 2000 0001 ไม่ได้

Data Transfer Instructions

- ทำ arithmetic operation กับ data ใน memory

  $\Rightarrow$ load data into registers $\triangleright$ do arithmetic $\triangleright$ store result from registers to memory

Instruction	Example	Meaning	Comments
load word	`lw $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Word from memory to register
store word	`sw $s1,20($s2)`	Memory[$s2 + 20] = $s1	Word from register to memory

ภาษาซี:

```
A[12] = h + A[8];
```

Data Transfer Instructions

Instruction	Example	Meaning	Comments
load word	`lw $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Word from memory to register
store word	`sw $s1,20($s2)`	Memory[$s2 + 20] = $s1	Word from register to memory

ภาษาซี:

```
A[12] = h + A[8];
```

- สมมติใช้ $s2 กับ $s3 เก็บ variables h และ base address ของ array A ตามลำดับ

Data Transfer Instructions

Instruction	Example	Meaning	Comments
load word	`lw $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Word from memory to register
store word	`sw $s1,20($s2)`	Memory[$s2 + 20] = $s1	Word from register to memory

ภาษาซี:

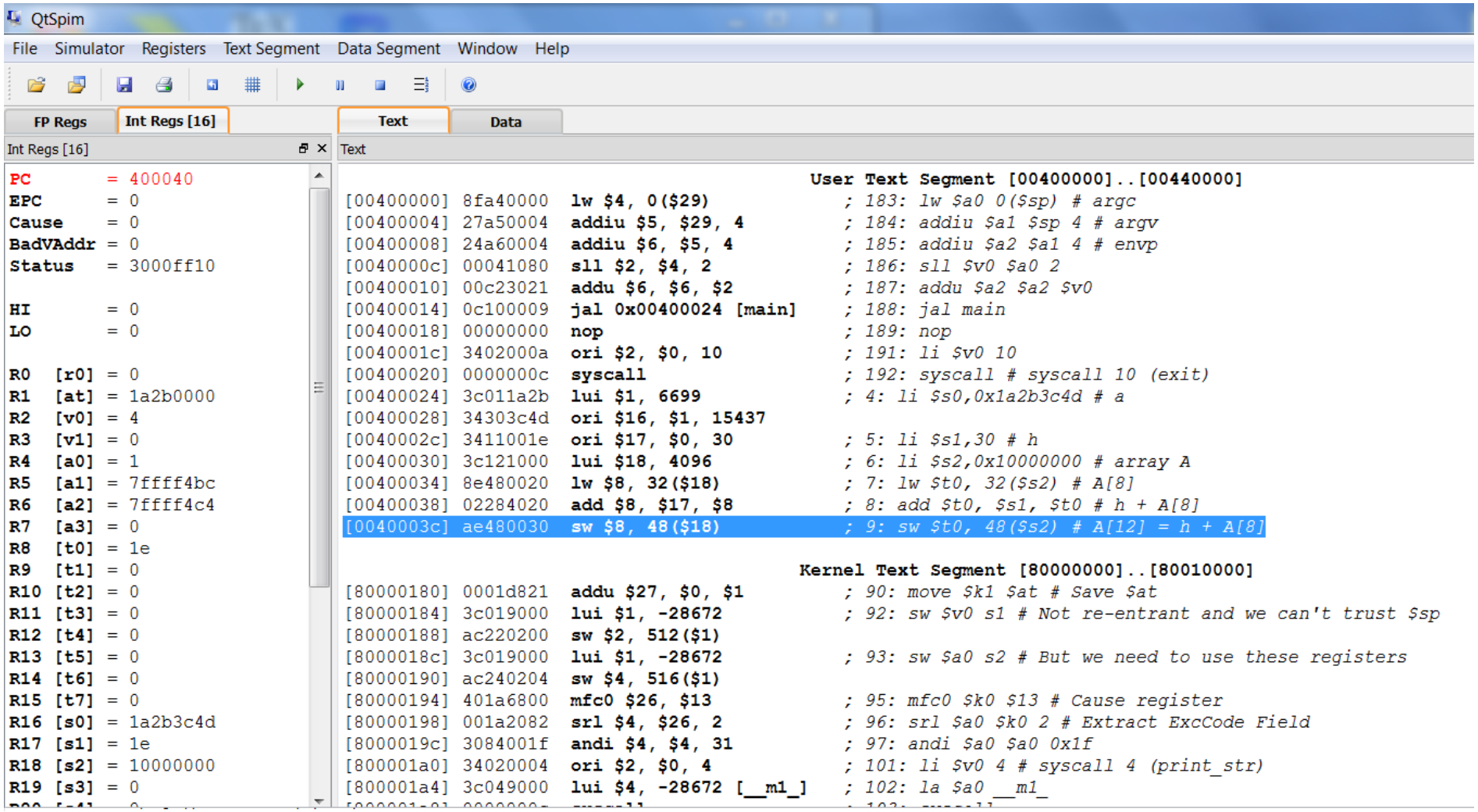
```
A[12] = h + A[8];
```

- สมมติใช้ $s2 กับ $s3 เก็บ variables h และ base address ของ array A ตามลำดับ

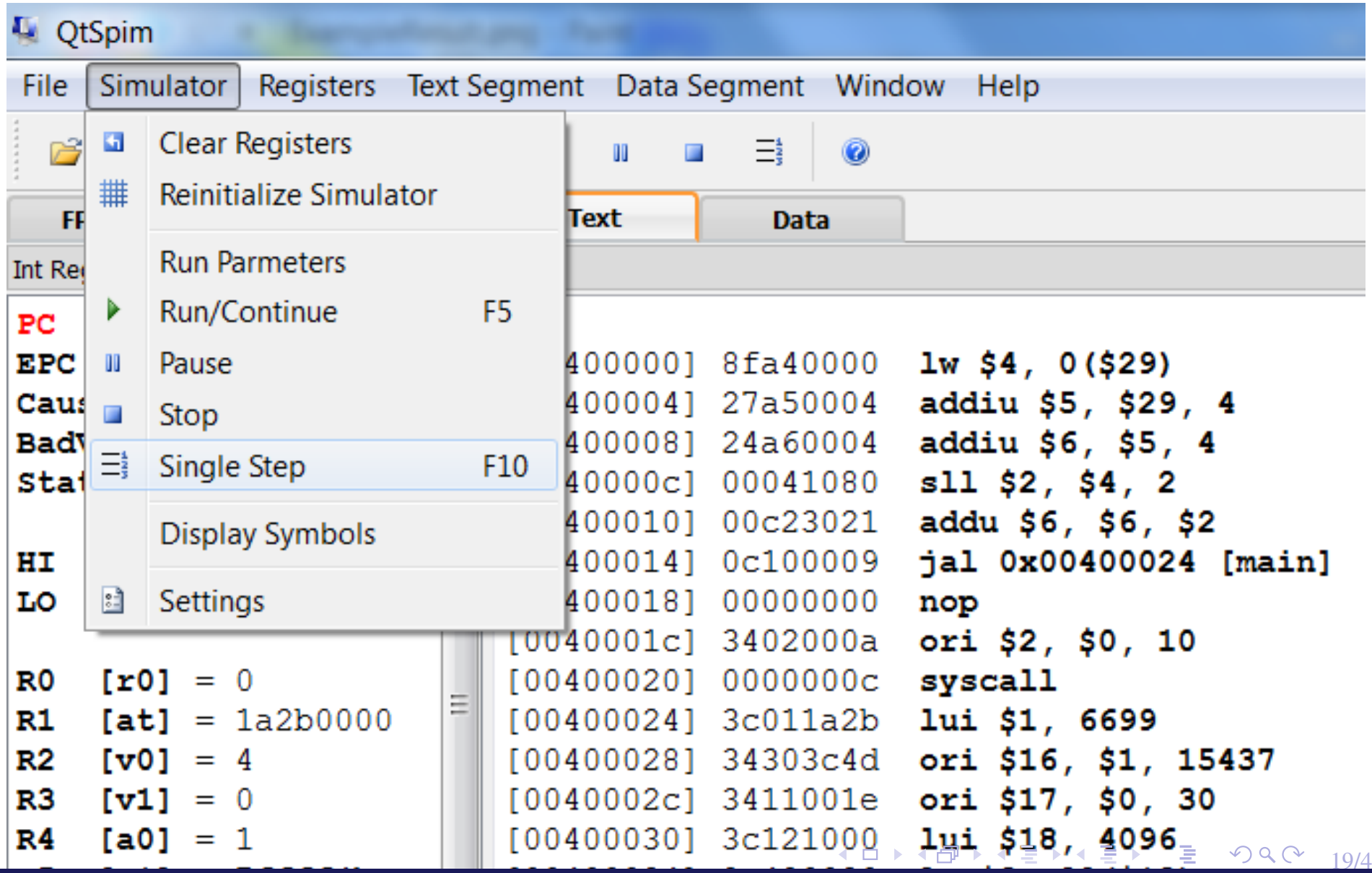
MIPS Assembly:

```
lw $t0, 32($s3)
add $t0, $s2, $t0
sw $t0, 48($s3)
```

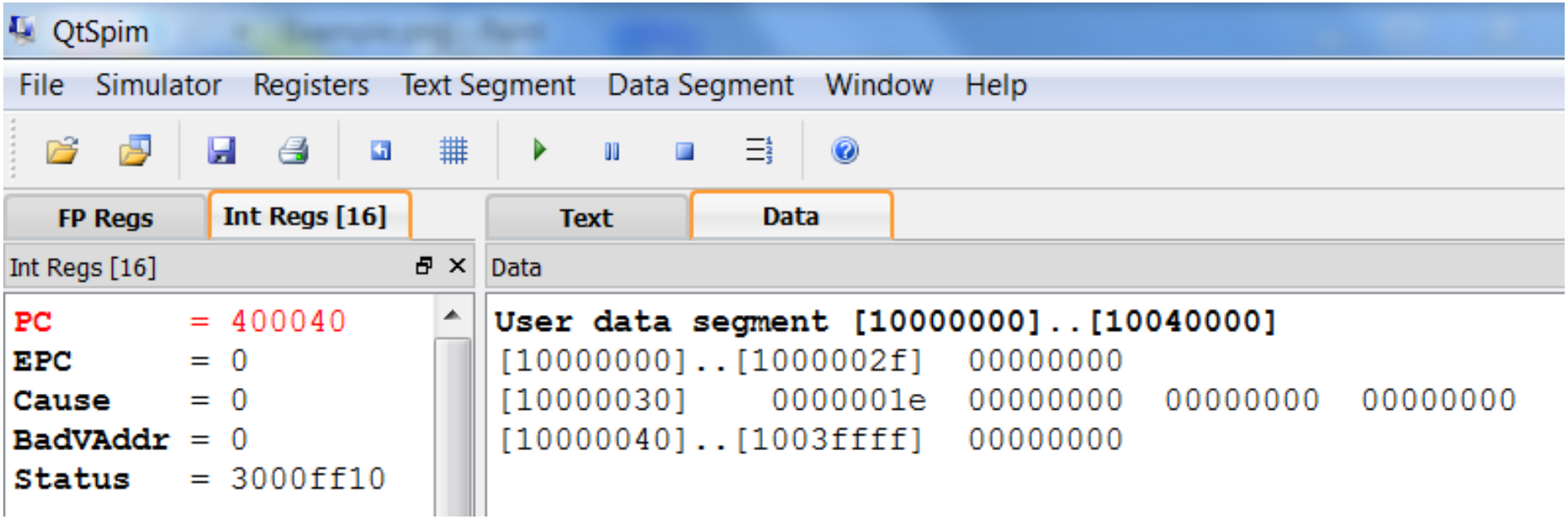
Simulating with SPIM



Simulating with SPIM



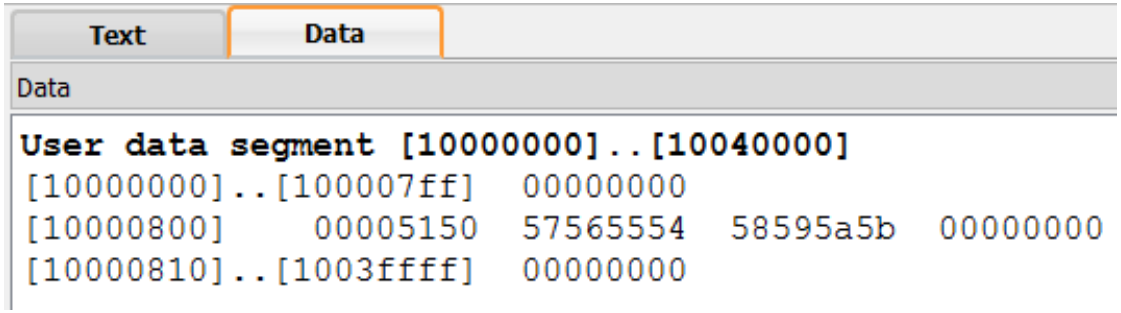
Simulating with SPIM



Simulating with SPIM

```
.data 0x10000800
.byte   0x50
.data 0x10000801
.byte   0x51
.data 0x10000804
.byte   0x54,0x55,0x56,0x57
.data 0x10000808
.word   0x58595A5B

.globl main
.text
main:   li $s5, 0x10000800
        lw $s1, 0x0(↩
           $s5)
stop:   j stop
```



Note: MIPS is big endian.

But, SPIM is little endian.

Registers v.s. Memory

- access registers เร็วกว่า access memory
- operation ที่ทำกับ memory ต้องทำผ่าน load กับ store
  ต้องใช้คำสั่งมากกว่า operation ที่ทำกับ registers
- compiler จะพยายามใช้ registers กับ variables ให้มากที่สุดเท่าที่ทำได้
  - เลือก variables ที่ใช้ไม่บ่อย เมื่อต้อง spill ไป memory

Constant or Immediate Operands

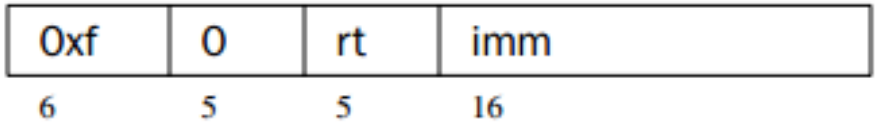
Instruction	Example	Meaning	Comments
add immediate	`addi $s1,$s2,20`	`$s1 = $s2 + 20`	Used to add constants

- บวกค่าใน register กับ ค่าคงที่ที่ระบุ
  addi $s3, $s3, 4
- ไม่มีคำสั่งลบค่าคงที่
  ใช้ บวกค่าเป็นลบแทนได้: addi $s2, $s1, -1
- register 0 ($zero) ถูก hard-wired เป็น ค่าคงที่ 0
  การย้ายข้อมูล, e.g., add $s0, $s1, $zero ≡ copy ข้อมูลจาก $s1 ไป $s0
- Design Principle 3: Make the common case fast
  - constant operand (ที่ค่าไม่ใหญ่) ถูกใช้บ่อยๆ

Load Upper Immediate

Load upper immediate

`lui rt, imm`



Load the lower halfword of the immediate `imm` into the upper halfword of register `rt`. The lower bits of the register are set to 0.

Load immediate

`li rdest, imm` *pseudoinstruction*

Move the immediate `imm` into register `rdest`.

Signed and Unsigned Numbers

- n-bit: $d_n \cdots d_2 d_1 d_0$ ในฐาน B
  มีค่า: $(d_n \times B^n) + \cdots + d_2 \times B^2) + (d_1 \times B^1) + (d_0 \times B^1)$
  - hexadecimal: 0xFA57
    $= (15 \times 16^3) + (10 \times 16^2) + (5 \times 16^1) + 7 = 64087$
  - binary: 0b1101
    $= (1 \times 2^3) + (1 \times 2^2) + 0 + 1 = 13$
- บิตที่มีค่ามากที่สุด เรียก “most significant bit” (msb)
- บิตที่มีค่าน้อยที่สุด เรียก “least significant bit” (lsb)

Representing Numbers

ตัวเลขในโลกจริง ไม่มีขนาดจำกัด
แต่ฮาร์ดแวร์ สำหรับ แทน ตัวเลข ในคอมพิวเตอร์ ขนาดจำกัด
MIPS word มี 32 บิต $\Rightarrow$ แทนข้อมูล ได้ 2^{32} แบบ.

$$
\begin{aligned}
00000000000000000000000000000000_{two} &= 0_{ten} \\
00000000000000000000000000000001_{two} &= 1_{ten} \\
00000000000000000000000000000010_{two} &= 2_{ten} \\
&\ldots \\
11111111111111111111111111111101_{two} &= 4,294,967,293_{ten} \\
11111111111111111111111111111110_{two} &= 4,294,967,294_{ten} \\
11111111111111111111111111111111_{two} &= 4,294,967,295_{ten}
\end{aligned}
$$

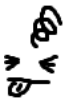
Signed Numbers

(signed) integer:

$$
\begin{aligned}
00000000000000000000000000000000_{two} &= 0_{ten} \\
00000000000000000000000000000001_{two} &= 1_{ten} \\
00000000000000000000000000000010_{two} &= 2_{ten} \\
&\dots \\
01111111111111111111111111111110_{two} &= 2,147,483,646_{ten} \\
01111111111111111111111111111111_{two} &= 2,147,483,647_{ten} \\
10000000000000000000000000000000_{two} &= -2,147,483,648_{ten} \\
10000000000000000000000000000001_{two} &= -2,147,483,647_{ten} \\
&\dots \\
11111111111111111111111111111110_{two} &= -2_{ten} \\
11111111111111111111111111111111_{two} &= -1_{ten}
\end{aligned}
$$

Signed Numbers

- most positive: $01111111111111111111111111111111 = 2,147,483,647_{ten}$
- most negative: $10000000000000000000000000000000 = -2,147,483,648_{ten}$
- 32-bit word: $-2^{n-1} = -2^{31} = -2,147,483,648_{ten}$
- แต่ทำ $-(-2^{n-1})$ ไม่ได้

  32-bit word ไม่มี $-(-2^{n-1}) = -(-2^{31}) \Rightarrow$ 

ลอง

```
int x;
x = 2147483647;
printf("x = %d\n",x);

x = x + 1;
printf("x = %d\n",x);
```

Two's Complement Signed Integer

- เพื่อความง่าย ดู 4-bit word
  - $3 + (-3) = 0 \Leftrightarrow 0011_{two} + x = 0000_{two}$, และ $x = (-3) = 1101_{two}$
  - n-bit word แสดง 2^n ไม่ได้
    เช่น 4-bit word: $2^n = 1\underline{0000}_{two}$ แสดงได้แค่ 0000_{two}
    ดังนั้น $0011_{two} + 1101_{two} = 0000_{two} \equiv 3 + (-3) = 0$

Two's Complement Signed Integer

- เพื่อความง่าย ดู 4-bit word
  - $3 + (-3) = 0 \Leftrightarrow 0011_{two} + x = 0000_{two}$, และ $x = (-3) = 1101_{two}$
  - n-bit word แสดง 2^n ไม่ได้
    เช่น 4-bit word: $2^n = 1\underline{0000}_{two}$ แสดงได้แค่ 0000_{two}
    ดังนั้น $0011_{two} + 1101_{two} = 0000_{two} \equiv 3 + (-3) = 0$
- ดูอีกมุม: $(-3) = 1101_{two}$

$$
\begin{array}{rrl}
 & 1101 & \\
= & 10000 & 2^n \\
- & 00011 & -x \\
\hline
= & 01101 & (\text{2's complement, } \hat{x}) \\
\hline
\end{array}
$$

Two's Complement Signed Integer

$$\begin{array}{rcl} & 10000 & 2^n \\ - & 00011 & -x \\ \hline = & 01101 & (\text{2's complement of } x) \end{array}$$

- $x + \hat{x} = 2^n$ แต่ n-bit word แสดง ออกมาเป็น 0

$$\underbrace{0011_{two}}_{3} + \underbrace{1101_{two}}_{-3} = \underbrace{(1)}_{\text{can't do}} \underbrace{0000_{two}}_{0}$$

"Two's complement gets its name from the rule that the unsigned sum of an n-bit number and its negative is 2^n; hence, the complement or negation of a two's complement number x is $2^n - x$."

Two's Complement Signed Integer: Shortcut

- Shortcut: 2's complement, $\hat{x} = \bar{x} + 1$

  $\bar{x}$: bitwise inversion of x, e.g., $x = 1101_{two} \Rightarrow \bar{x} = 0010_{two}$

Two's Complement Signed Integer: Shortcut

- Shortcut: 2's complement, $\hat{x} = \bar{x} + 1$

  $\bar{x}$: bitwise inversion of x, e.g., $x = 1101_{two} \Rightarrow \bar{x} = 0010_{two}$

$$\begin{aligned} \hat{x} &= 2^n - x \\ &= (2^n - 1) - x + 1 \end{aligned}$$

- แต่ $(2^n - 1) = 11...11_{two}$

  เช่น 4-bit word: $2^4 - 1 = 15 = 1111_{two}$

- และ $11..11_{two} - x = \bar{x}$

  เช่น $1111 - 1011 = 0100$

- ดังนั้น $(2^n - 1) - x = \bar{x}$ และ $\hat{x} = \bar{x} + 1$

Sign Extension

8-bit word

16-bit word

$$
\begin{aligned}
2 &= 00000010_{two} \\
12 &= 00001100_{two} \\
-2 &= 11111110_{two} \\
-12 &= 11110100_{two}
\end{aligned}
$$

Sign Extension

8-bit word

$$
\begin{aligned}
2 &= 00000010_{two} \\
12 &= 00001100_{two} \\
-2 &= 11111110_{two} \\
-12 &= 11110100_{two}
\end{aligned}
$$

16-bit word

$$
\begin{aligned}
2 &= 0000000000000010_{two} \\
12 &= 0000000000001100_{two} \\
-2 &= 1111111111111110_{two} \\
-12 &= 1111111111110100_{two}
\end{aligned}
$$

Sign Extension

8-bit word			16-bit word		
2	$=$	00000010_{two}	2	$=$	0000000000000010_{two}
12	$=$	00001100_{two}	12	$=$	0000000000001100_{two}
-2	$=$	11111110_{two}	-2	$=$	1111111111111110_{two}
-12	$=$	11110100_{two}	-12	$=$	1111111111110100_{two}

ถ้าต้องการ แปลงจาก 8-bit word ไปเป็น 16-bit word

$\Rightarrow$ แค่เติม bit ที่เพิ่มมา ด้วยค่าของ msb

เช่น

$0\,0001100_{two} \Rightarrow \underline{0000000}00001100_{two}$;

$1\,1110100_{two} \Rightarrow \underline{1111111}11110100_{two}$

Representing Instructions in the Computer

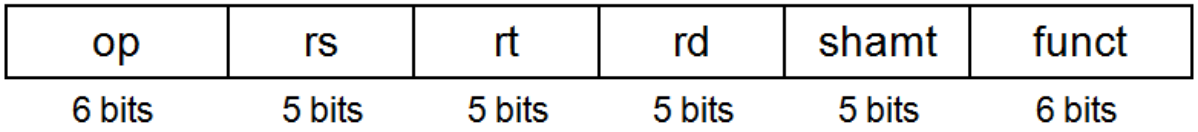
- instructions ถูกเก็บในรูป binary เรียกว่า “machine code”
- MIPS instructions
  - encoded as 32-bit instruction words
  - มีไม่กี่ formats (3 formats) $\Rightarrow$ regularity

Name	Register number	Usage
$zero	0	The constant value 0
$v0–$v1	2–3	Values for results and expression evaluation
$a0–$a3	4–7	Arguments
$t0–$t7	8–15	Temporaries
$s0–$s7	16–23	Saved
$t8–$t9	24–25	More temporaries
$gp	28	Global pointer
$sp	29	Stack pointer
$fp	30	Frame pointer
$ra	31	Return address

MIPS Instruction Formats

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

MIPS R-Format Instructions



Instruction fields

- op: operation code (opcode)
- rs: first source register number
- rt: second source register number
- rd: destination register number
- shamt: shift amount (00000 for now)
- funct: function code (extends opcode)

R-format Example

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

add $t0, $s1, $s2

R-type	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$$0000|0010|0011|0010|0100|0000|0010|0000_2 = 02324020_{16}$$

$$\underbrace{\text{add \$t0, \$s1, \$s2}}_{\text{assembly}} \Rightarrow \underbrace{02324020_{16}}_{\text{machine code}}$$

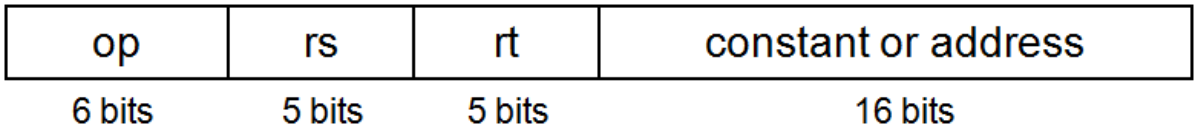
Hexadecimal

- Base 16
- ดูสะดวกกว่า base 2
- 4 bits = 1 hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

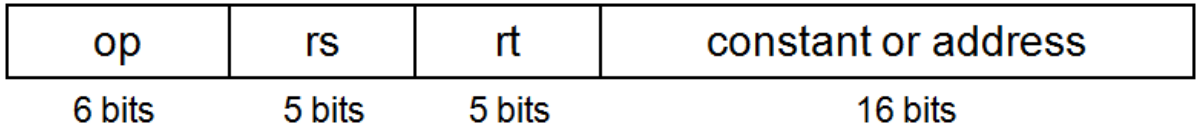
- ตัวอย่าง: eca8 6420 (hexadecimal)
  = 1110 1100 1010 1000 0110 0100 0010 0000 (binary)

MIPS I-Format Instructions



- Immediate arithmetic and load/store instructions
  - immediate arithmetic, e.g., addi
  - rs: register number (the first register)
  - rt: register number (the second/write register)
  - constant: 16 bits ในรูป 2's complement
    มีค่าได้ในช่วง ตั้งแต่ -2^{15} ถึง $2^{15} - 1$
- Design Principle 4: Good design demands good compromises
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

I-Format: Immediate Arithmetic Example



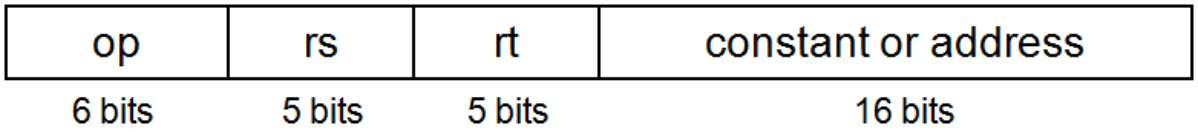
addi $s0, $t0, 18

addi	$t0	$s0	18
001000	01000	10000	0000000000010010

$$0010|0001|0001|0000|0000|0000|0001|0010_2 = 21100012_{16}$$

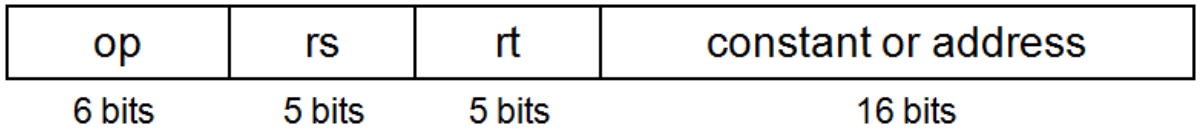
$$\underbrace{\text{addi } \$s0,\ \$t0,\ 18}_{\text{assembly}} \Rightarrow \underbrace{21100012_{16}}_{\text{machine code}}$$

MIPS I-Format Instructions



- load/store, e.g., lw, sw
  - rs: base-address register number
  - rt: destination or source register number
  - address: offset added to base address in rs

I-Format: Load Example



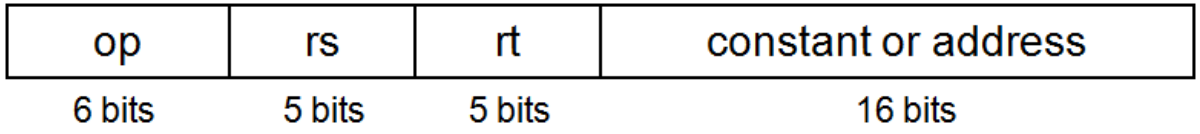
lw $t0, 32($s4)

lw	$s4	$t0	32
100011	10100	01000	0000000000100000

$$1000|1110|1000|1000|0000|0000|0010|0000_2 = 8e880020_{16}$$

$$\underbrace{\text{lw } \$t0,\ 32(\$s4)}_{\text{assembly}} \Rightarrow \underbrace{8e880020_{16}}_{\text{machine code}}$$

I-Format: Store Example



sw $t0, 32($s4)

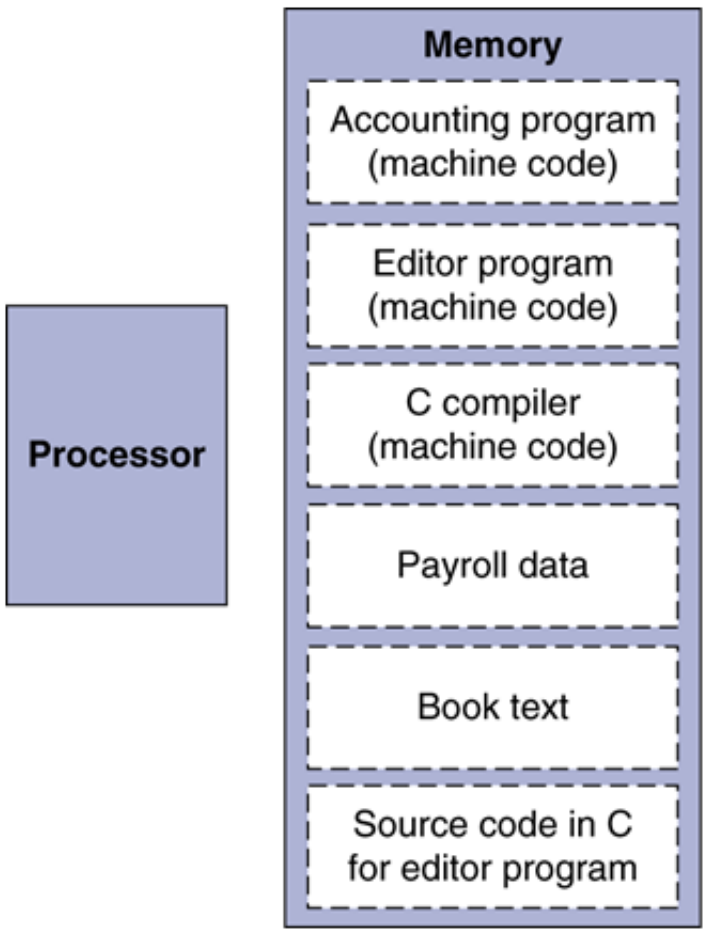
sw	$s4	$t0	32
101011	10100	01000	0000000000100000

$1010|1110|1000|1000|0000|0000|0010|0000_2 = ae880020_{16}$

$\underbrace{\text{sw } \$t0,\ 32(\$s4)}_{\text{assembly}} \Rightarrow \underbrace{ae880020_{16}}_{\text{machine code}}$

Stored Program Computers

The BIG Picture



- ทั้ง intructions และ data เก็บ เป็น binary ใน memory
- Programs can operate on programs
  E.g., compilers
- Binary compatibility allows compiled programs to work on different computers
  $\Rightarrow$ Standardized ISAs

Closing note

- อ่าน P&H5e, 2.1-2.5.

